FinSense AI — Project Context for Gemini CLI

What this project is

FinSense AI is a hackathon MVP for SBI's GFF Hackathon 2026. It is a multi-agent banking assistant with three specialised agents sharing one customer data layer:

- **AcquireAgent** — conversational onboarding, KYC, account opening
- **AdoptAgent** — detects behaviour patterns, nudges customers toward products
- **EngageAgent** — life-event detection, proactive engagement, anomaly alerts

All three agents are orchestrated by a single **FinSense Router** that classifies intent and hands off to the right agent. Agents share a **Customer Intelligence Store** (a simple JSON/SQLite store acting as the "memory graph" from the pitch deck).

Constraints for this MVP (read before generating code)

- This is a 30-day hackathon build. DO NOT integrate real SBI systems — Finacle, CKYC, and UPI are all MOCKED with local FastAPI endpoints that return realistic fake JSON. Always label mocked endpoints clearly in code comments.
- DO NOT use Kafka, Pinecone, or paid infra. Use:
  - SQLite (not Postgres) for storage, unless told otherwise
  - A local JSON file replayed on a timer to simulate "transaction events" instead of Kafka
  - Plain keyword/file-based retrieval instead of Pinecone for RAG (a `data/product_docs/` folder of .md/.txt files agents can grep/search)
- LLM calls go through a single wrapper function `call_llm()` in `backend/app/llm.py` so the model provider (Gemini, Claude, Kimi) can be swapped via `.env` without touching agent code.
- Keep agent logic in plain Python functions with clear tool-calling structure (function name, description, parameters, handler) — don't pull in LangGraph/AutoGen unless explicitly asked; the goal is code a judge can read end-to-end in 10 minutes.

Tech stack (actual, not aspirational)

- Backend: Python, FastAPI, SQLite, Uvicorn
- Frontend: React + Vite + Tailwind
- LLM: Gemini API (primary), with provider-swap support for Claude/Kimi
- OCR: Tesseract (pytesseract) for Aadhaar/PAN extraction
- No Docker required for local dev; keep it `pip install -r requirements.txt` simple

File/folder conventions

- backend/app/agents/ → one file per agent, exposes `handle(message, customer_id)`
- backend/app/tools/ → tool functions agents can call (typed, docstring per tool)
- backend/app/mock_sbi/ → fake CBS/CKYC/UPI endpoints, clearly commented as mocks
- backend/app/memory/ → shared customer intelligence store (read/write helpers)
- backend/app/data/ → seed data: customers.json, transactions.csv, product_docs/

Coding style

- Type hints everywhere in Python
- Every tool function has a docstring explaining what it does (these get shown to the LLM)
- No silent failures — log and surface errors, this is a live demo
- Prefer small, runnable increments. After generating each file, tell me how to test it.

Current phase

We are in Phase 1: backend skeleton + AcquireAgent. Do not build AdoptAgent or EngageAgent yet unless asked.